MATA KULIAH DATA SCIENCE

PERTEMUAN 12

Jaringan Saraf Tiruan (Neural Networks) - Bagian 1

Sub-CPMK 10

D4 Teknologi Rekayasa Perangkat Lunak - Politeknik Negeri Madiun

Tujuan Pembelajaran

SUB-CPMK 10

1	Mahasiswa memahami arsitektur Neural Network, fungsi aktivasi, backpropagation, dan regularisasi

Peta Materi Pertemuan

1 Membentuk Model Non-Linear

2 Arsitektur Jaringan Saraf

3 Fungsi Aktivasi

4 Keterbatasan Model Linier dan Fixed Basis Function

5 Dari Neuron Biologis ke Neuron Artifisial

6 Universal Approximation dan Mengapa 'Deep'?

7 Membangun Neural Network dengan PyTorch

8 Melatih Model dengan Autograd dan Optimizer

MATERI 1

Membentuk Model Non-Linear

Model linier seperti regresi linier dan logistik terbatas dalam memodelkan hubungan yang kompleks. Neural Network mengatasi keterbatasan ini dengan menyusun banyak unit komputasi sederhana secara berlapis untuk menghasilkan fungsi non-linier yang jauh lebih fleksibel.

- Neural Network terinspirasi dari cara kerja neuron biologis, meski secara matematis jauh lebih sederhana
- Kombinasi banyak fungsi linier + fungsi aktivasi non-linier menghasilkan kemampuan memodelkan pola kompleks
- Neural Network dapat mendekati (approximate) hampir semua jenis fungsi jika arsitekturnya cukup besar (universal approximation)

MATERI 2

Arsitektur Jaringan Saraf

Arsitektur dasar Neural Network terdiri dari lapisan input, satu atau lebih lapisan tersembunyi (hidden layer), dan lapisan output, yang masing-masing terdiri dari neuron/unit yang saling terhubung.

- Input layer: menerima fitur data mentah
- Hidden layer: melakukan transformasi bertahap terhadap representasi data
- Output layer: menghasilkan prediksi akhir (nilai kontinu untuk regresi, probabilitas untuk klasifikasi)
- Setiap koneksi antar-neuron memiliki bobot (weight) yang dipelajari selama pelatihan

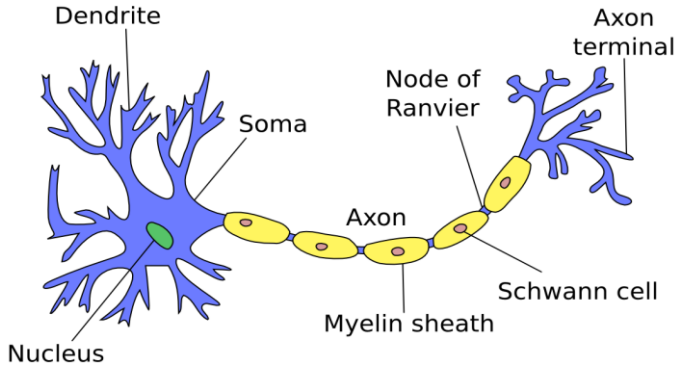
Neuron

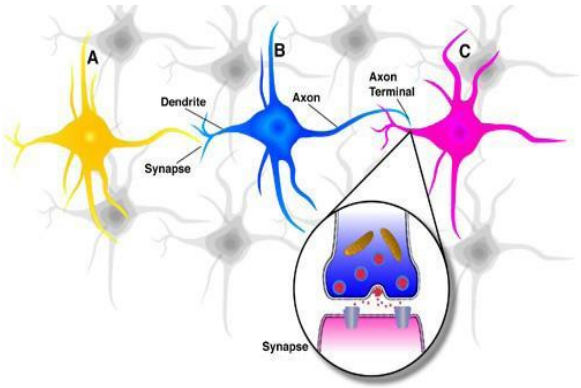
1935834

The Structure of a Biological Neuron

1935834

- Simple building block: **Neuron**
- Dendrite: Collecting signals from other neurons.
- Soma (cell body): Spatial summation and processing.
- Axon: Transmitting signals to dendrites of other cells.
- Synapse: Neurons use synaptic connections to exchange information.
  - Synapses can be either **excitatory** or **inhibitory** in strength.

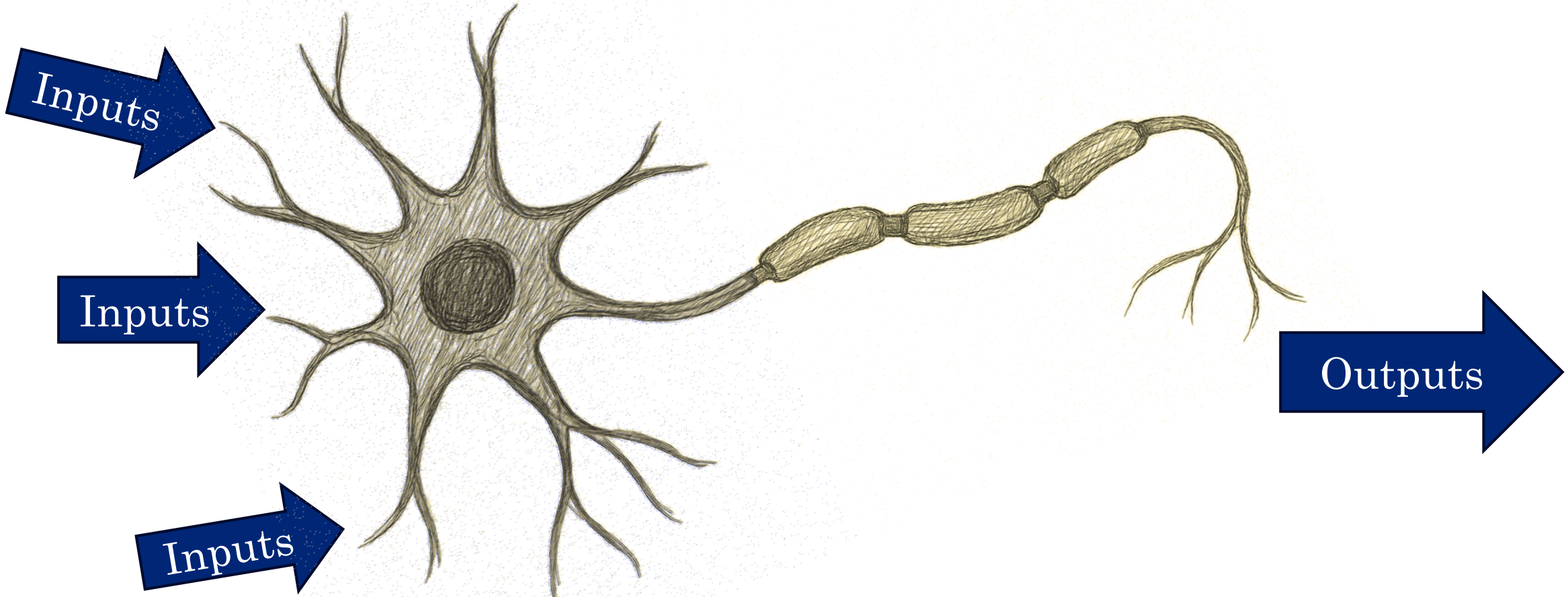




Starting with a Single Neuron

1935834

Simulate the **firing of a neuron** as a **weighted combination of its inputs.**



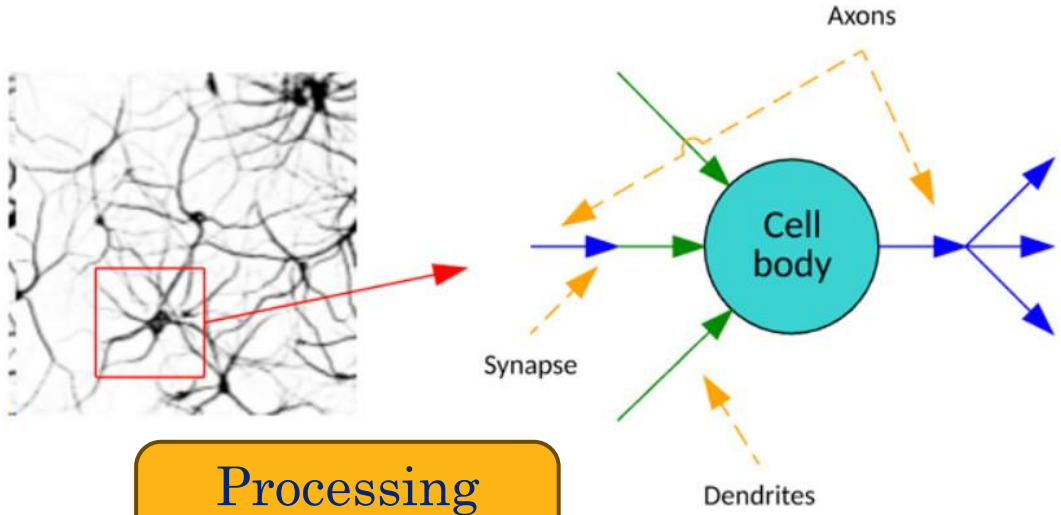
CC BY NC SA

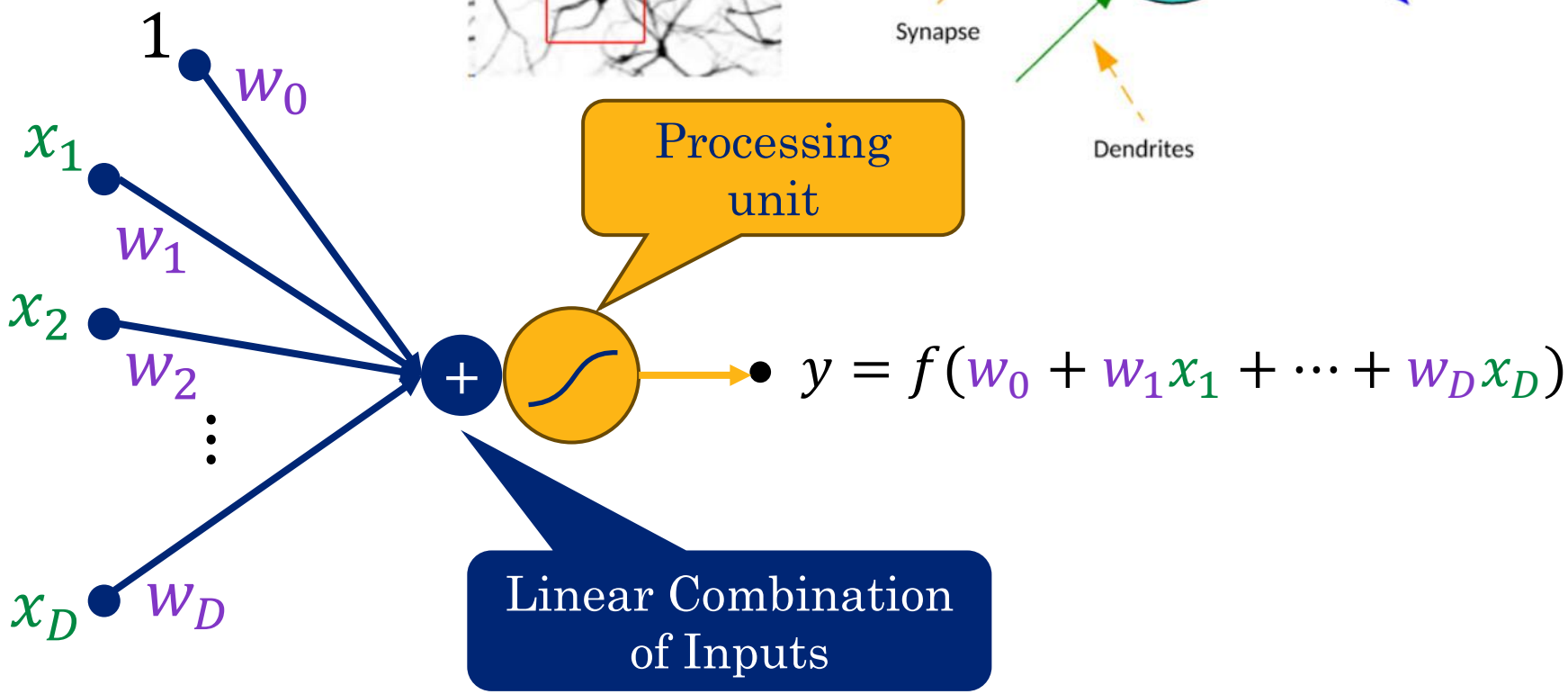
From Biological to Artificial Neurons

1935834

- Biological neuron contains
  - Inputs
  - Summation of inputs
  - Processing unit
  - Output

- Artificial neuron





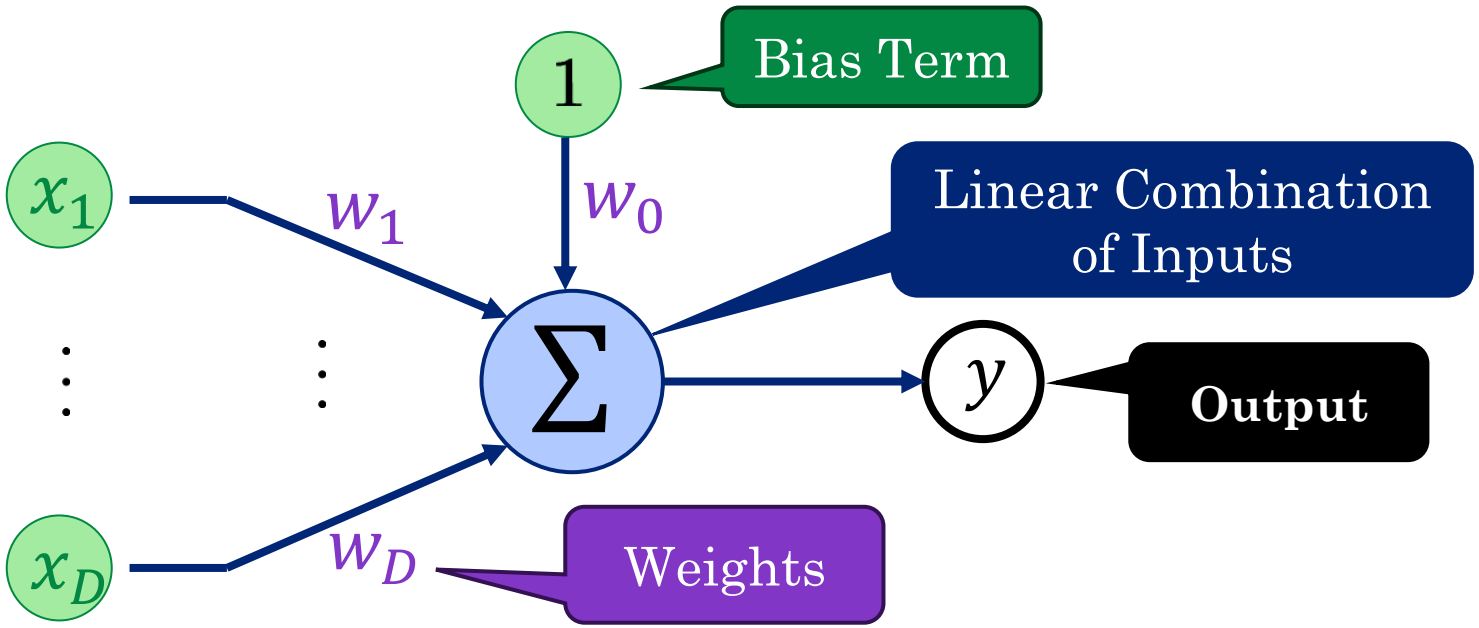
CC BY NC SA

Linear Regression in a Summation Unit

1935834

- Graphical representation
  - Inputs are x_i values entering the summation unit.
  - Connections are weighted by parameters.

$$y = \vec{w}.\vec{x} = \sum_{d=0}^{D} w_d x_d$$



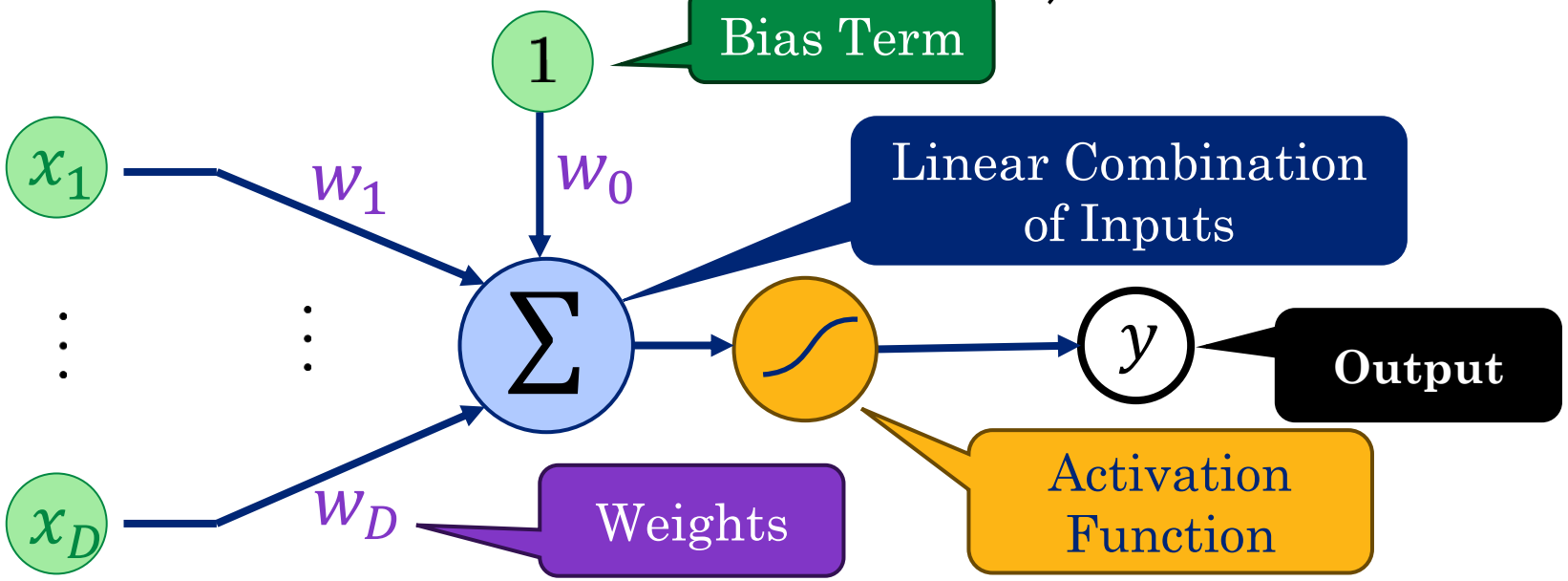
CC BY NC SA

Neuron – Activation Function

1935834

- An artificial neuron is a **summation unit** + **an activation function**.
- The weighted summation is passed to the activation function to calculate the neuron's activation.

$$y = f(\vec{w}.\vec{x}) = f\left(\sum_{d=0}^{D} w_d x_d\right)$$



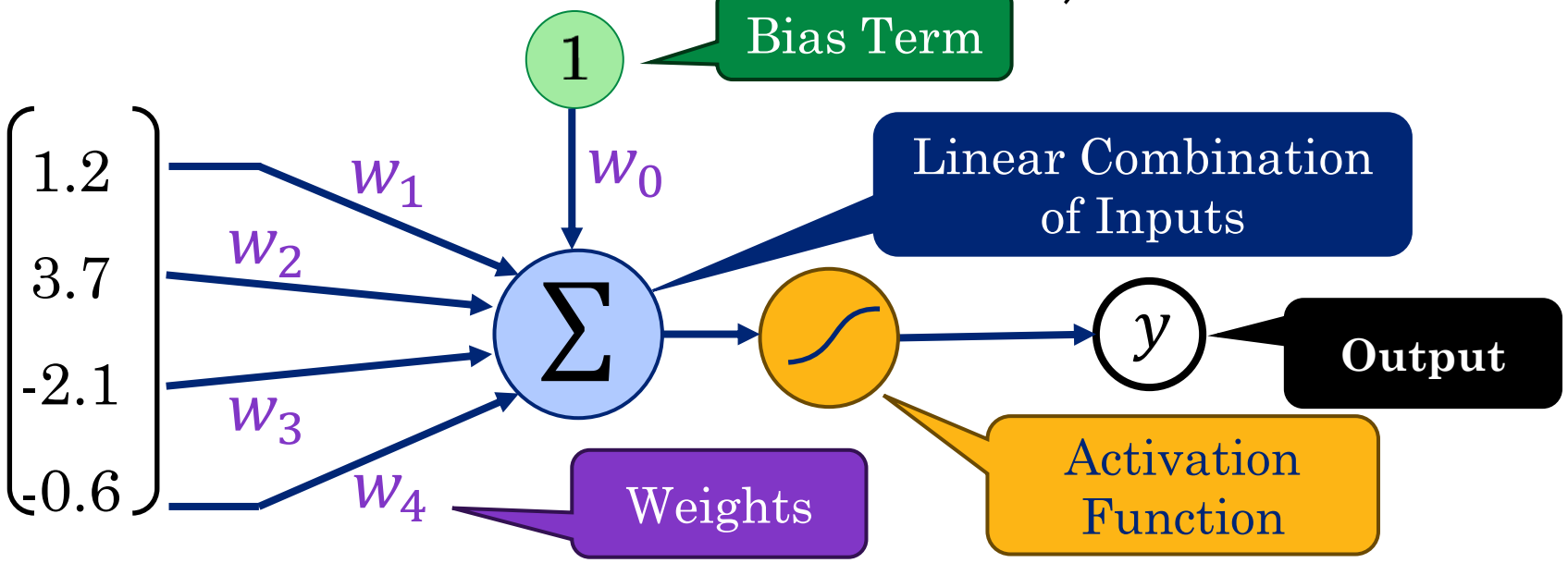
CC BY NC SA

A Simple Neuron

1935834

- An artificial neuron is a **summation unit** + **an activation function**.
- The weighted summation is passed to the activation function to calculate the neuron's activation.

$$y = f(\vec{w}.\vec{x}) = f\left(\sum_{d=0}^{D} w_d x_d\right)$$



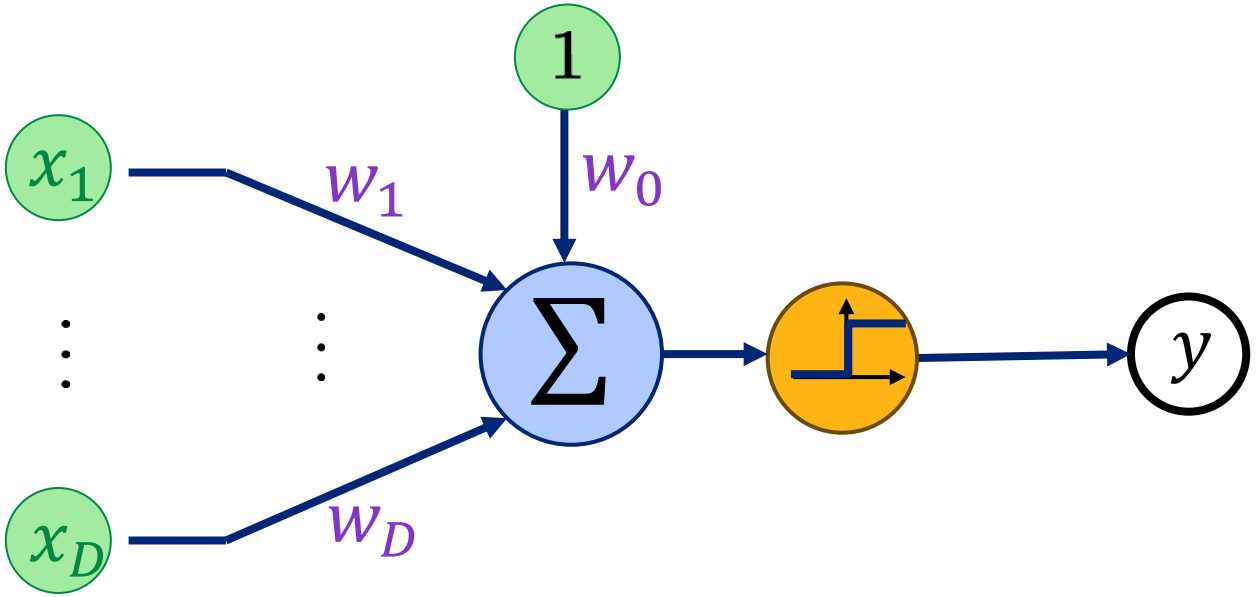
CC BY NC SA

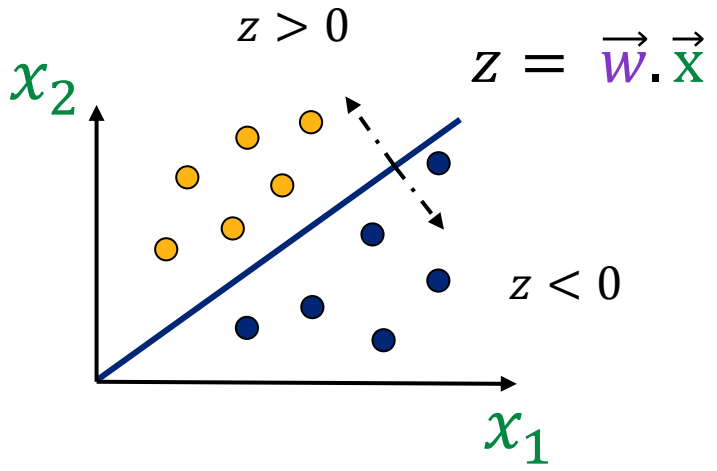
A Neuron with Step Function as Activation

1935834

- Assume the activation function to be a **step function**.
  - If the weighted combination of inputs is **negative**, the output is **0** → neuron is not activated.
  - If the weighted combination of inputs is **positive**, the output is **1** → neuron is activated.

$$f(z) = \begin{cases} 1 & \text{if } z > 0 \\ 0 & \text{if } z < 0 \end{cases}$$



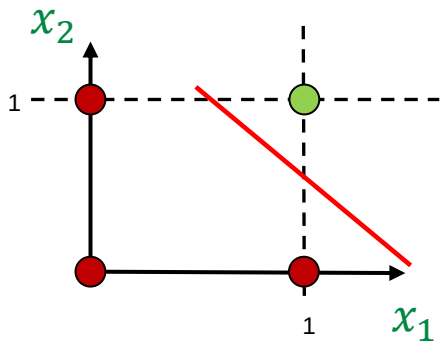


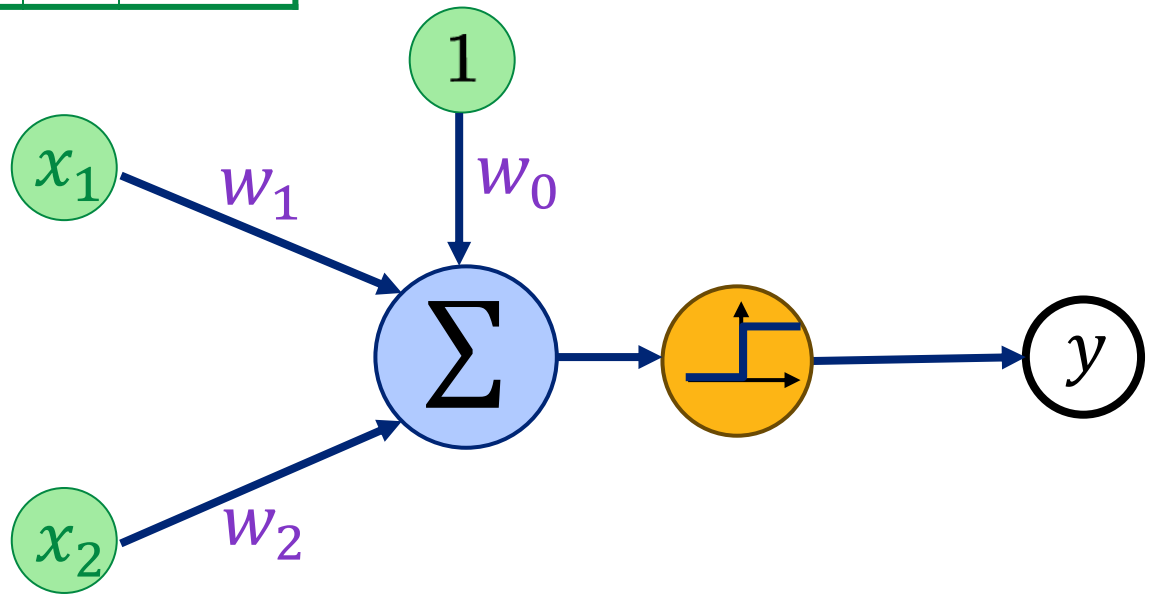
CC BY NC SA

Functional Completeness - AND

1935834

x_1	x_2	y
1	0	0
1	1	1
0	0	0
0	1	0



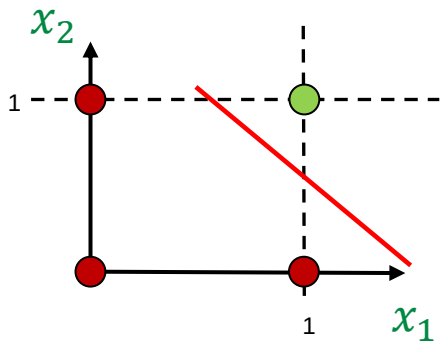


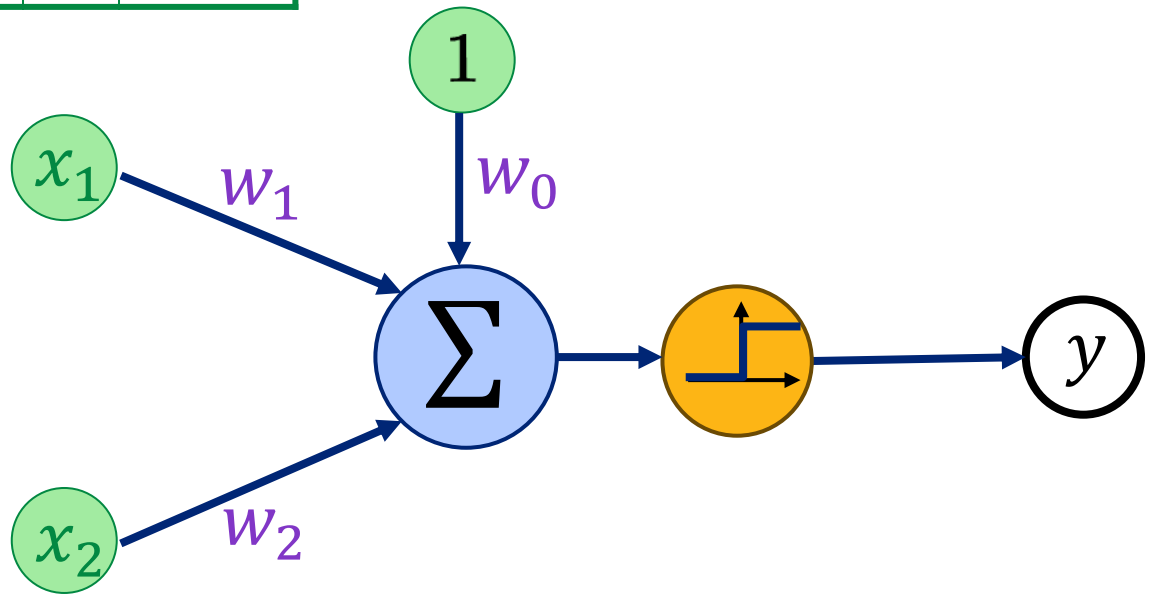
CC BY NC SA

Functional Completeness - AND

1935834

x_1	x_2	y
1	0	0
1	1	1
0	0	0
0	1	0



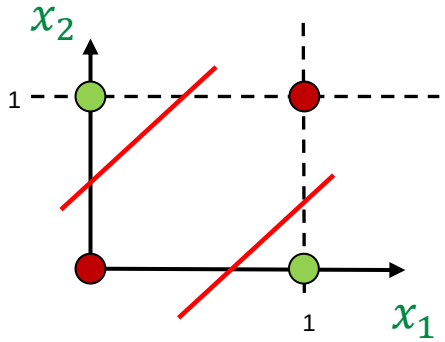


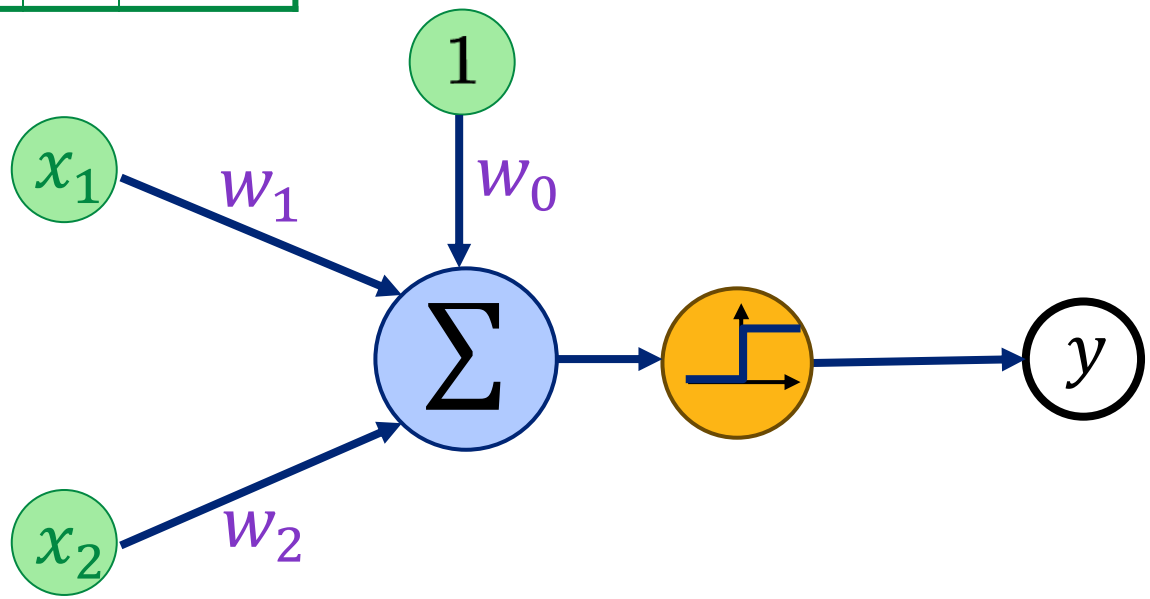
CC BY NC SA

Functional Completeness - XOR

1935834

x_1	x_2	y
1	0	1
1	1	0
0	0	0
0	1	1





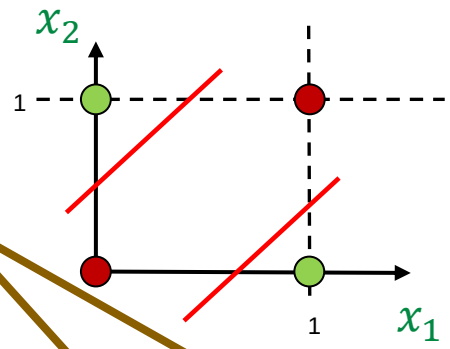
CC BY NC SA

Functional Completeness - XOR

1935834

x_1	x_2	y
1	0	1
1	1	0
0	0	0
0	1	1

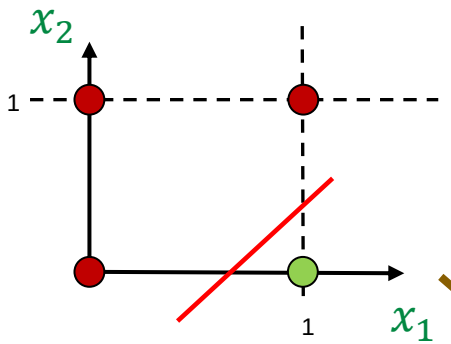
$x_1 XOR\ x_2$



Non-linear Decision Boundary
(No linear separability)

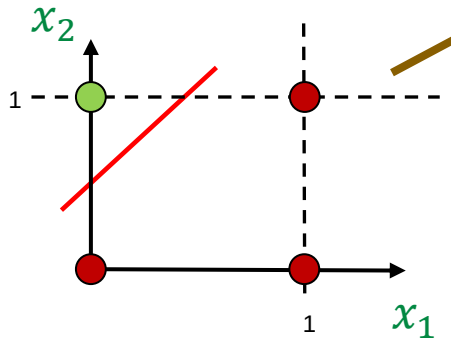
$x_1 AND \sim x_2$

x_1	x_2	y
1	0	1
1	1	0
0	0	0
0	1	0

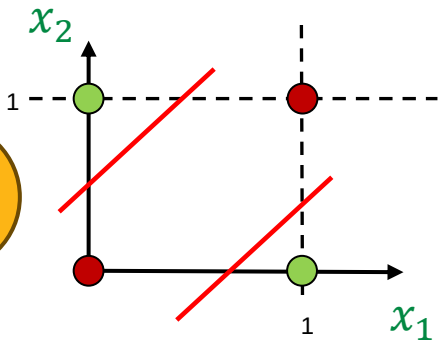


$\sim x_1 AND\ x_2$

x_1	x_2	y
1	0	0
1	1	0
0	0	0
0	1	1



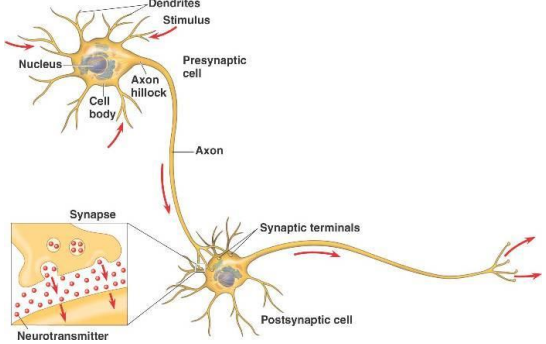
OR



Neural Network

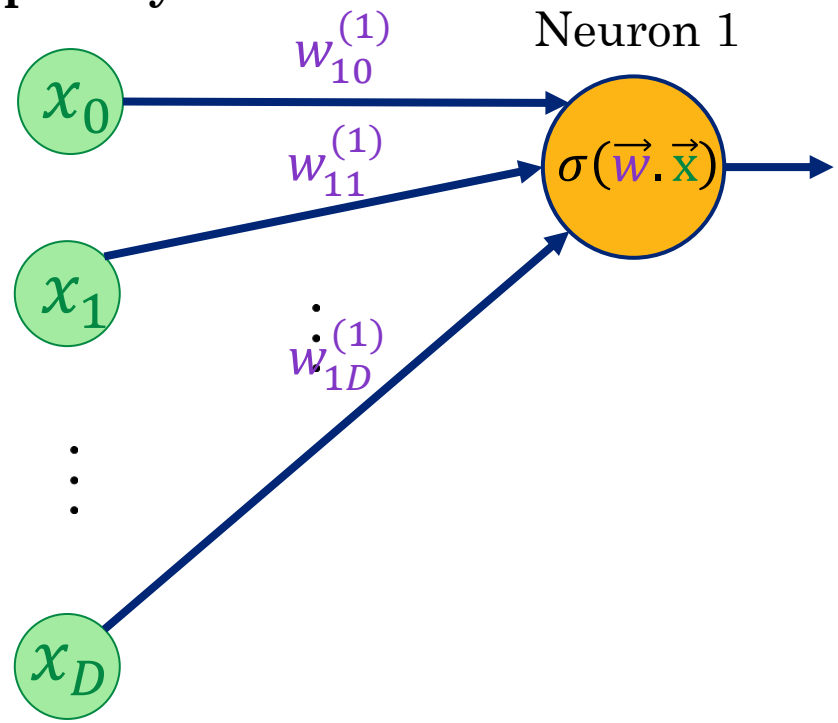
1935834

Neural Network



1935834

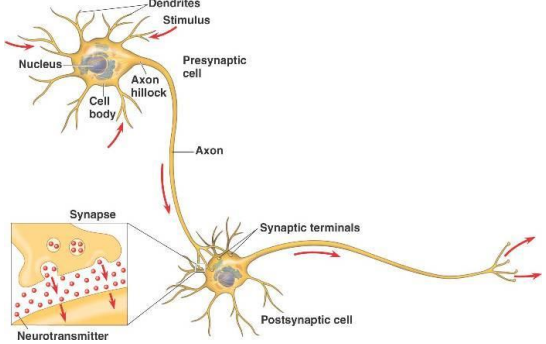
Input layer



Superscript (1) indicates that corresponding parameters are in the first 'layer' of the network.

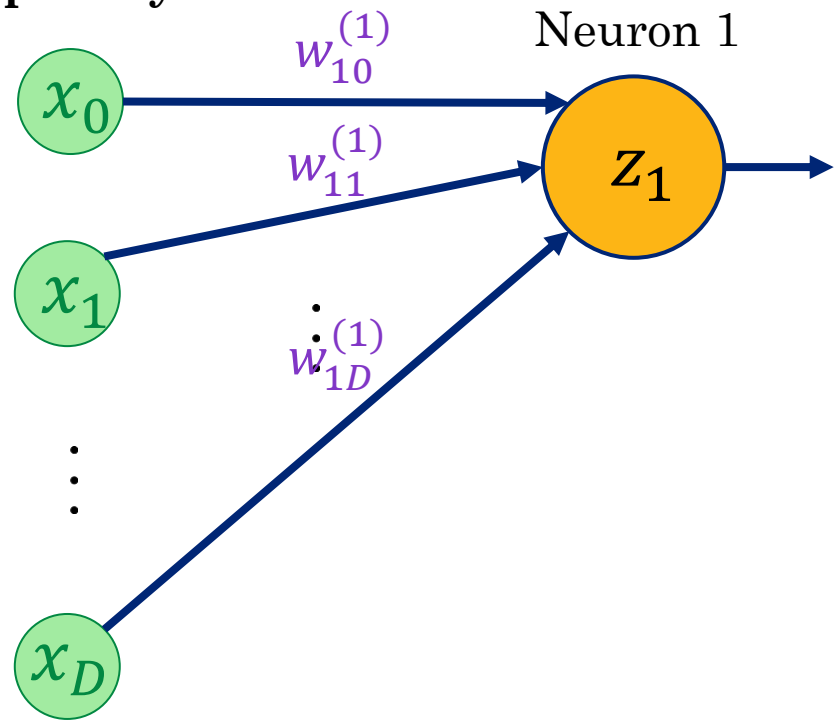
CC BY NC SA

Neural Network



1935834

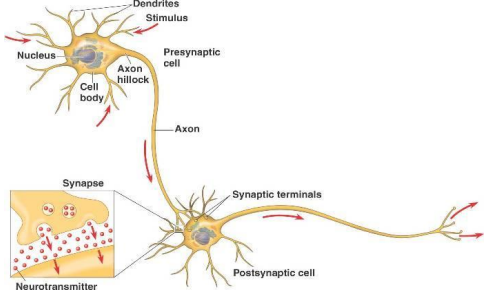
Input layer



Superscript (1) indicates that corresponding parameters are in the first 'layer' of the network.

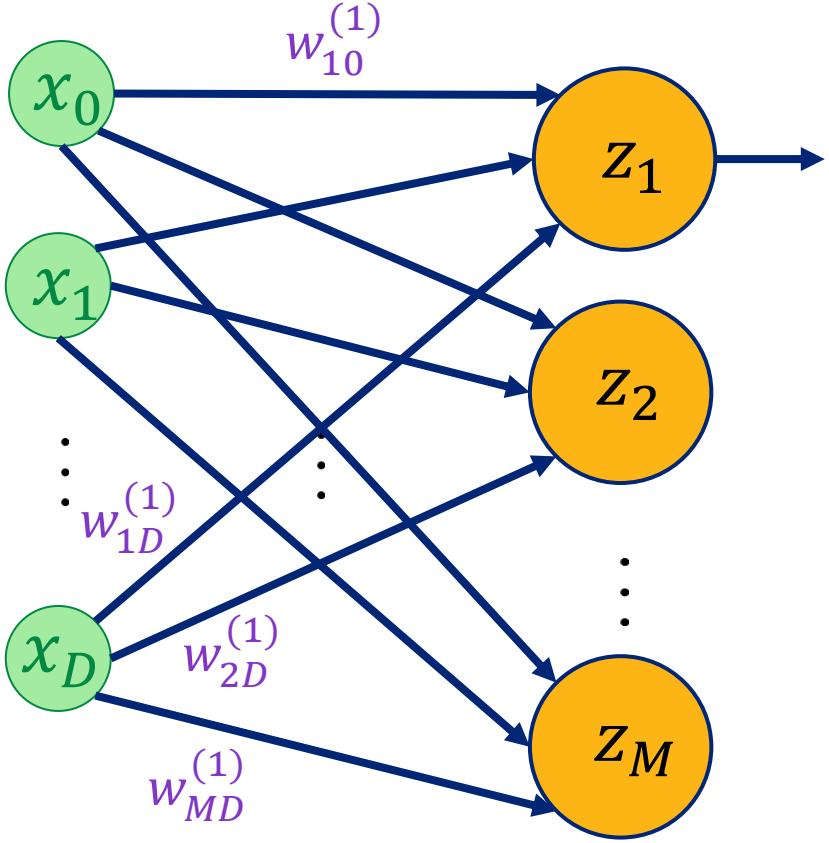
CC BY NC SA

Neural Network



1935834

Input layer **Hidden layer**



Superscript (1) indicates that corresponding parameters are in the first 'layer' of the network.

Activations (M linear combination of inputs)

$$a_j^{(1)} = \sum_{i=1}^{D} w_{ji}^{(1)} x_i + w_{j0}^{(1)} \qquad j \in \{1, \dots, M\}$$

Bias Term

Pre-activation or logits

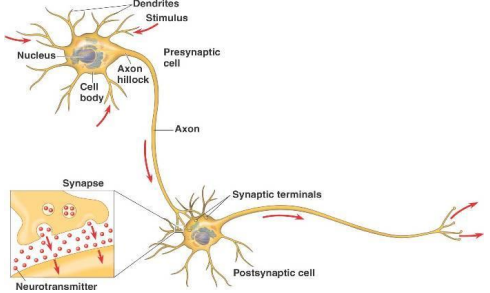
Weights

$$z_j^{(1)} = h(a_j^{(1)}) \qquad j \in \{1, \dots, M\}$$

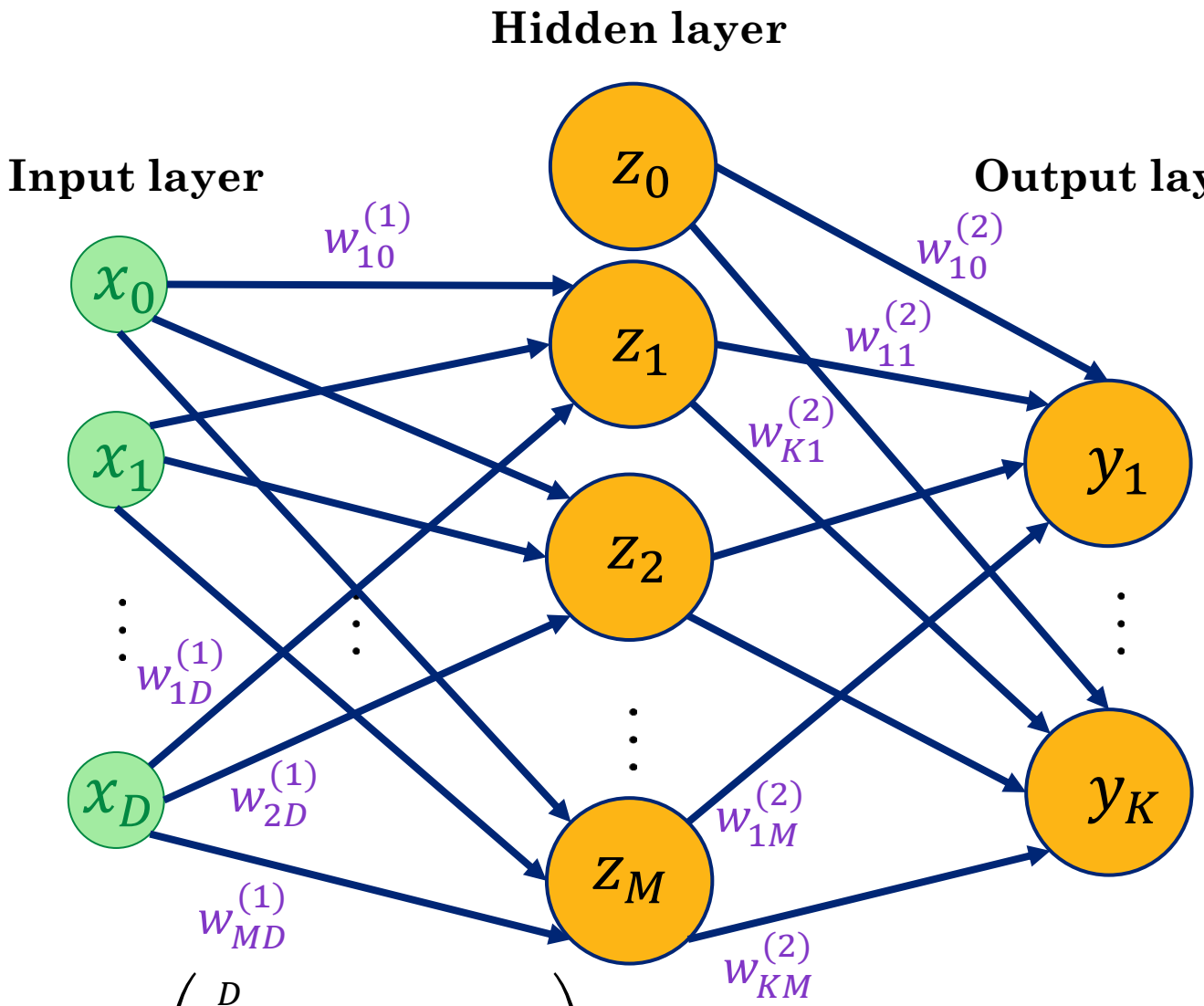
Output of basis functions

Activation Function

CC BY NC SA



1935834



$$a_k^{(2)} = \sum_{j=1}^{M} w_{kj}^{(2)} z_j^{(1)} + w_{k0}^{(2)} \qquad k \in \{1, \dots, K\}$$

$$y(x, w) = f\left(a_k^{(2)}\right)$$

$$= f\left(\sum_{j=1}^{M} w_{kj}^{(2)} z_j^{(1)} + w_{k0}^{(2)}\right)$$

$$= f\left(\sum_{j=1}^{M} w_{kj}^{(2)} h\left(\sum_{i=1}^{D} w_{ji}^{(1)} x_i + w_{j0}^{(1)}\right) + w_{k0}^{(2)}\right)$$

$$= f\left(W^{(2)} h\left(W^{(1)} \mathrm{x}\right)\right)$$

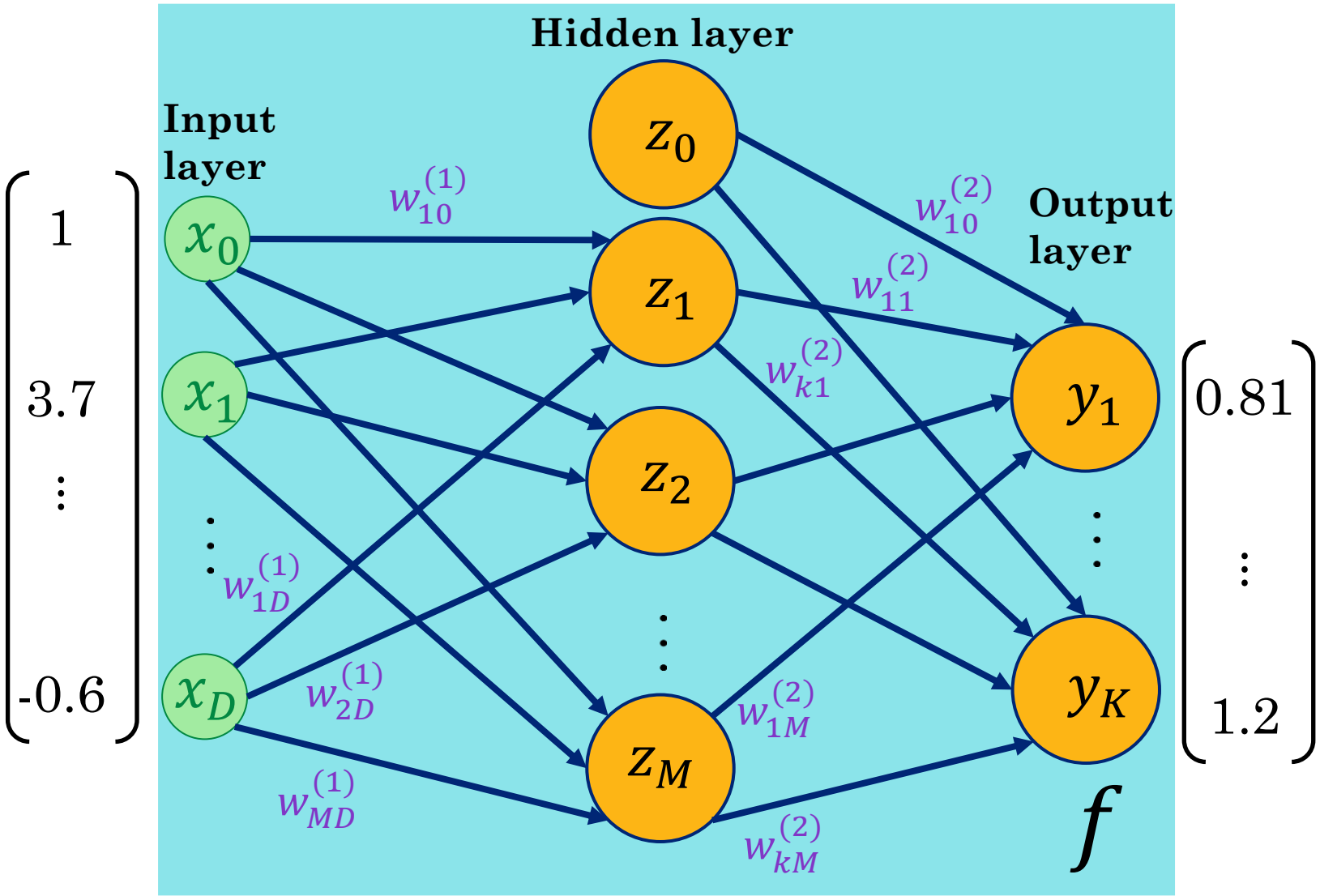
$$z_j^{(1)} = h\left(\sum_{i=1}^{D} w_{ji}^{(1)} x_i + w_{j0}^{(1)}\right) \qquad j \in \{1, \dots, M\}$$

Now, let's add output layer.

What about the intercept?

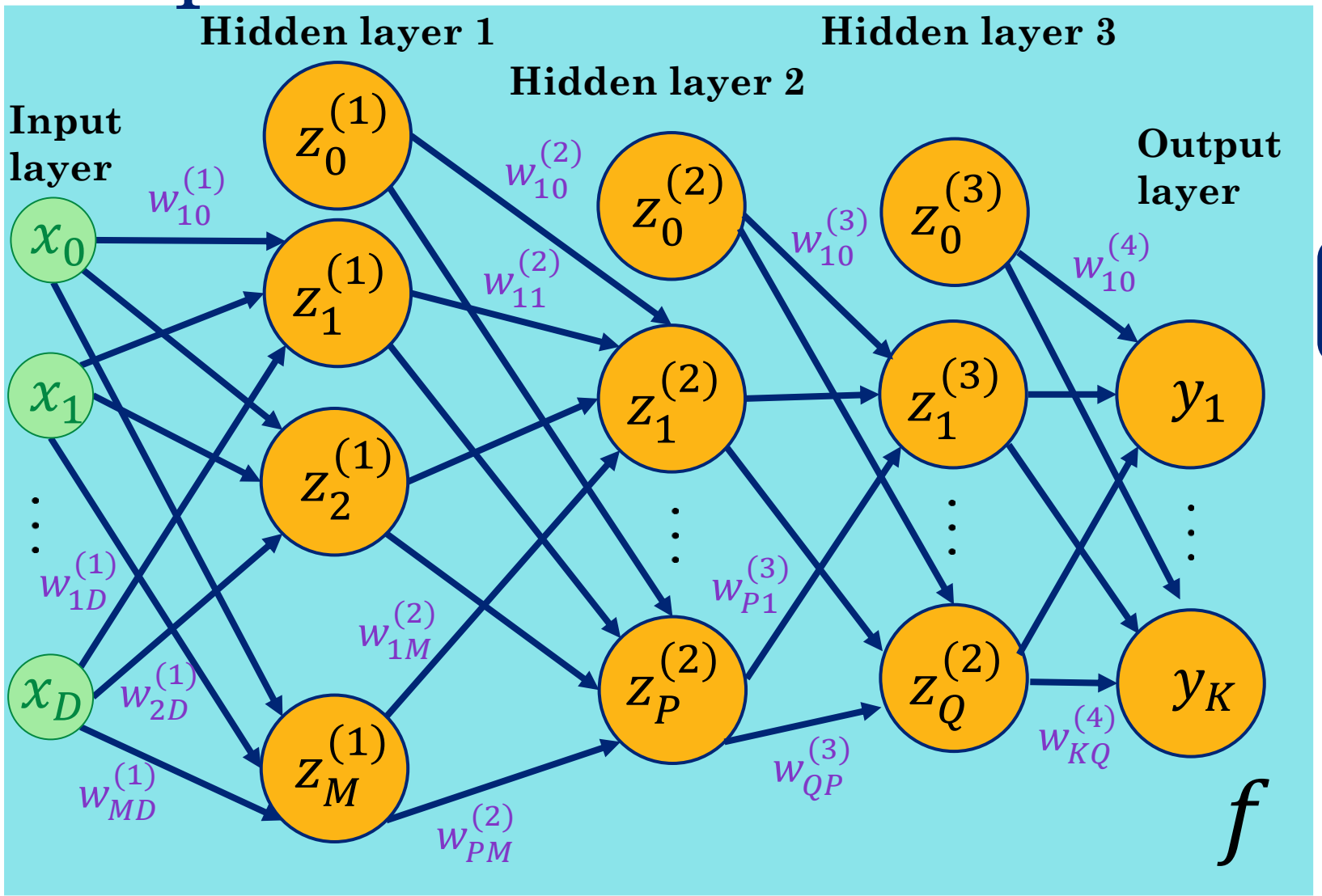
CC BY NC SA

Neural Network



1935834

Deep Neural Network



$l \in \{1, \ldots, L\}$

$$z^{(l)} = h^{(l)}\left(W^{(l)} z^{(l-1)}\right)$$

Activation in layer l

Weights in layer l

$z^{(0)} = \mathrm{x}$

$z^{(L)} = \mathrm{y}$

1935834

Activation Functions

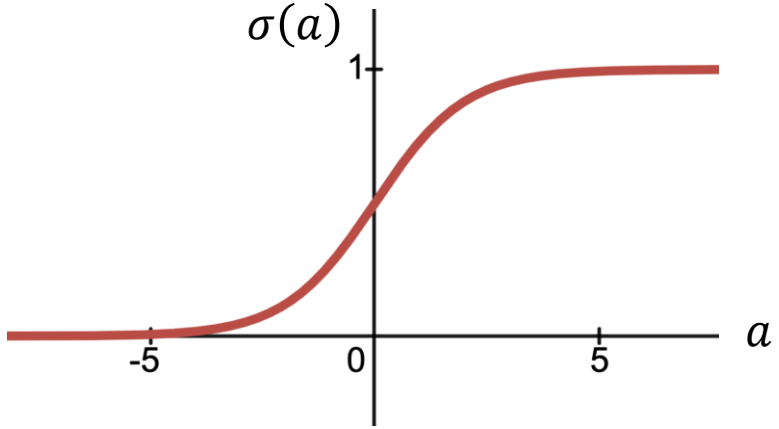
1935834

Activation Functions

1935834

- The simplest option for a hidden unit activation function is the identity function.
  - But it is redundant to have a hidden layer if all activations are identity.
- Simple non-linear differentiable activation function is sigmoid.

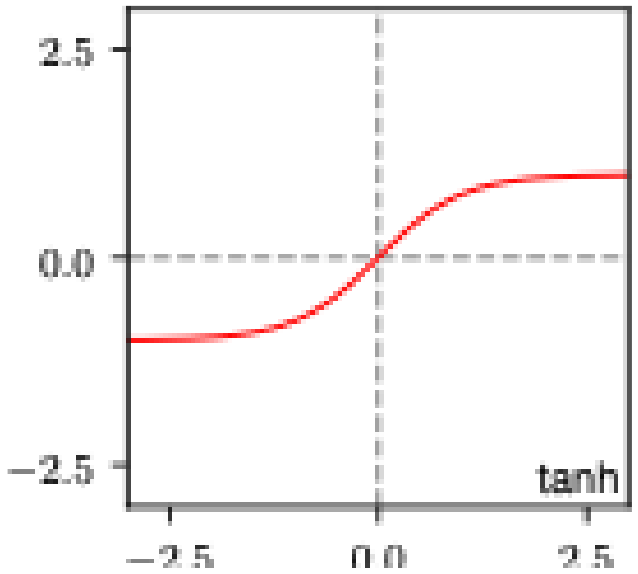
$$\sigma(a) = \frac{1}{1 + e^{-a}}$$

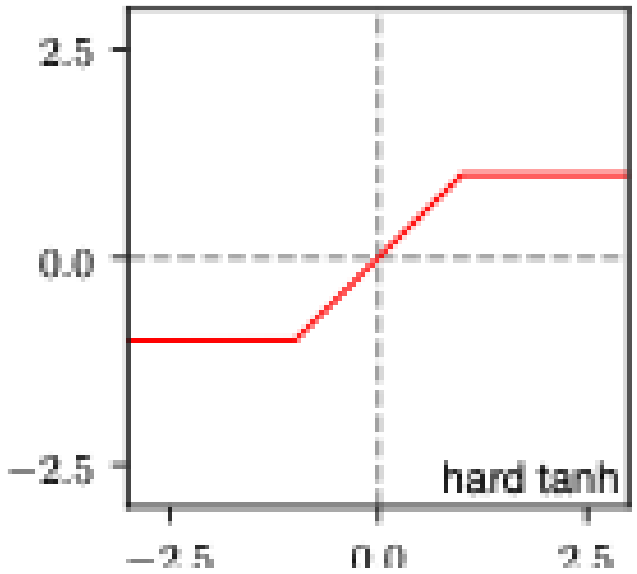


CC BY NC SA

Other Activation Functions

1935834





$$\tanh(a) = 2\sigma(2a) - 1$$

$$= \frac{e^{a} - e^{-a}}{e^{a} + e^{-a}}$$

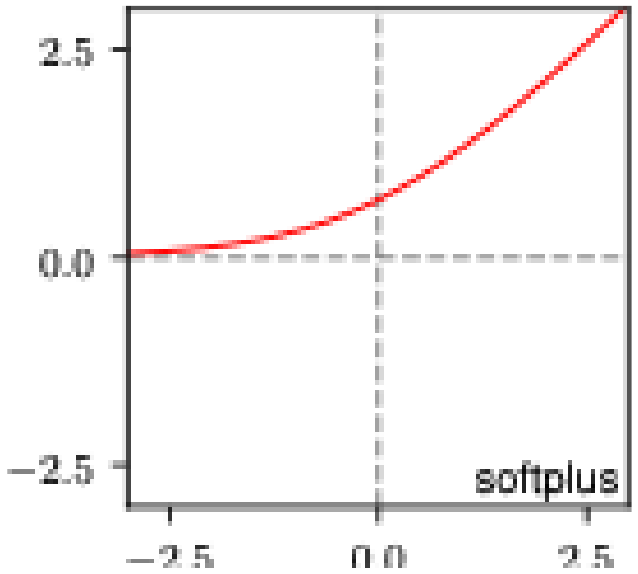
$$\mathrm{h}(a) = \max(-1, \min(1, a))$$

One issue with tanh and sigmoid family action function is that gradients go to 0 when magnitude of activations are large.

We need an activation that has non-zero gradient at least when activations are large positive values.

Other Activation Functions

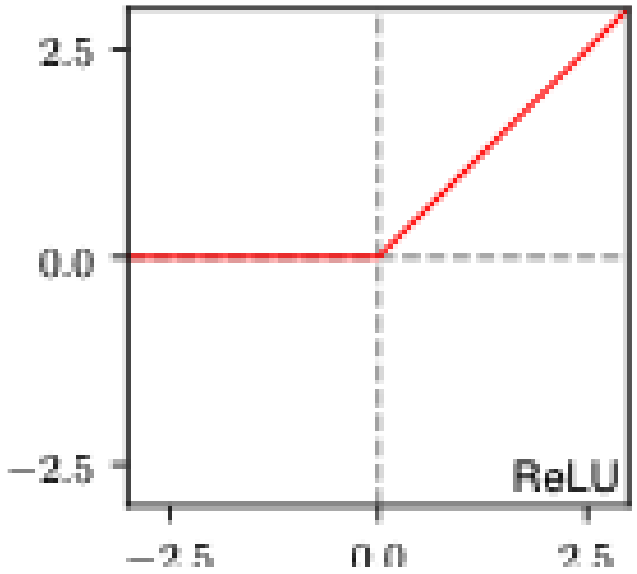
1935834



$h(a) = \ln(1 + \exp(a))$

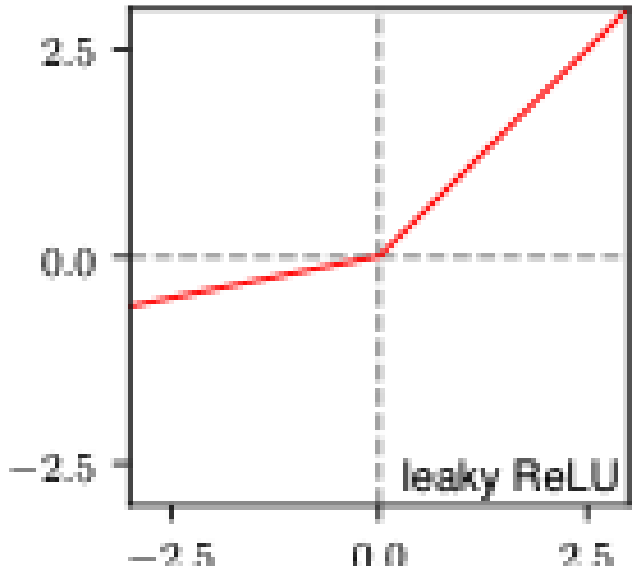
When $a \gg 1$, $h(a) \simeq a$

Also called soft ReLU



$h(a) = \max(0, a)$

Neurons with negative activations will receive "no error signal"



$h(a) = \max(0, a) + \alpha \min(0, a)$

MATERI 3

Fungsi Aktivasi

Fungsi aktivasi memberikan non-linieritas pada Neural Network. Tanpa fungsi aktivasi non-linier, tumpukan banyak lapisan linier tetap akan setara dengan satu lapisan linier tunggal.

- Sigmoid: menghasilkan output antara 0 dan 1, umum digunakan pada output klasifikasi biner
- ReLU (Rectified Linear Unit): f(x) = max(0, x), populer pada hidden layer karena efisien dan mengurangi masalah vanishing gradient
- Tanh: mirip sigmoid tapi rentang output -1 hingga 1
- Softmax: digunakan pada output layer untuk klasifikasi multi-kelas

MATERI 4

Keterbatasan Model Linier dan Fixed Basis Function

Model linier (termasuk regresi dengan fungsi basis tetap) memiliki keterbatasan mendasar: kompleksitasnya bergantung pada dimensi data, bukan pada struktur sebenarnya dari data tersebut.

- Contoh klasik keterbatasan model linier: masalah XOR - tidak dapat dipisahkan oleh satu garis linier (non-linearly separable)
- Pada dimensi tinggi, data sering kali sebenarnya terletak pada manifold berdimensi rendah yang tersembunyi di dalam ruang berdimensi tinggi
- Jika model dapat 'mengikuti' manifold tersebut, kompleksitas model bergantung pada dimensi manifold (kecil), bukan dimensi data mentah (besar)
- Neural Network belajar basis fungsi langsung dari data (data-dependent basis) dan menyesuaikan diri dengan manifold data, alih-alih menggunakan basis fungsi yang dirancang secara manual (fixed/hand-crafted)

MATERI 5

Dari Neuron Biologis ke Neuron Artifisial

Neural Network terinspirasi dari struktur neuron biologis, meskipun neuron artifisial jauh lebih sederhana secara matematis dibanding neuron sungguhan.

- Neuron biologis terdiri dari dendrite (menerima sinyal), soma (memproses/menjumlahkan sinyal), axon (meneruskan sinyal), dan synapse (penghubung antar-neuron)
- Neuron artifisial meniru proses ini sebagai: unit penjumlahan berbobot (weighted sum) dari input, ditambah bias, lalu dilewatkan ke fungsi aktivasi
- Fungsi aktivasi step (tangga) adalah versi paling sederhana: neuron 'aktif' (output 1) jika kombinasi input positif, dan 'tidak aktif' (output 0) jika negatif
- Satu neuron dengan fungsi aktivasi step saja sudah bisa mereplikasi gerbang logika sederhana (AND, OR), namun tidak bisa mereplikasi XOR - inilah yang mendorong kebutuhan menyusun banyak neuron menjadi jaringan berlapis

MATERI 6

Universal Approximation dan Mengapa 'Deep'?

Universal Approximation Theorem menyatakan bahwa Neural Network dengan satu hidden layer saja (dengan neuron yang cukup banyak) secara teoritis dapat merepresentasikan hampir semua fungsi kontinu. Namun, dalam praktiknya, jaringan yang 'dalam' (deep, banyak lapisan) lebih disukai dibanding jaringan yang 'lebar' (fat, satu lapisan besar).

- Analogi dengan rangkaian logika: dua lapisan gerbang logika secara teori bisa merepresentasikan fungsi Boolean apa pun, tapi menggunakan lebih banyak lapisan seringkali menghasilkan rangkaian yang jauh lebih sederhana (lebih sedikit gerbang/parameter)
- Hal serupa berlaku pada Neural Network: menyusun beberapa hidden layer (deep) dapat merepresentasikan fungsi kompleks dengan jauh lebih sedikit parameter dibanding satu hidden layer yang sangat lebar
- Inilah alasan utama di balik istilah 'Deep Learning' - kedalaman (jumlah lapisan) memberikan efisiensi representasi, bukan sekadar lebar jaringan

MATERI 7

Membangun Neural Network dengan PyTorch

PyTorch adalah framework paling banyak digunakan untuk membangun dan melatih Neural Network, dengan dukungan differensiasi otomatis (automatic differentiation) dan akselerasi perangkat keras.

- torch.tensor adalah struktur data dasar PyTorch, mirip numpy.ndarray, namun mendukung penempatan di GPU dan pelacakan gradien secara otomatis
- nn.Module adalah kelas dasar untuk seluruh model/layer di PyTorch - method __init__ mendefinisikan parameter model, method forward mendefinisikan alur komputasi model
- nn.Parameter digunakan untuk membungkus tensor agar dapat dilacak dan diperbarui secara otomatis selama pelatihan
- PyTorch menyediakan ratusan modul layer siap pakai (nn.Linear, nn.Softmax, dll.) yang dapat dirangkai untuk membangun arsitektur Neural Network

MATERI 8

Melatih Model dengan Autograd dan Optimizer

PyTorch menggunakan automatic differentiation (autograd) untuk menghitung gradien loss terhadap seluruh parameter model secara otomatis dan efisien, kemudian optimizer menerapkan aturan pembaruan (seperti Adam) berdasarkan gradien tersebut.

- Alur dasar pelatihan: reset gradien (zero_grad) → hitung prediksi model → hitung loss → panggil loss.backward() untuk menghitung gradien → optimizer.step() untuk memperbarui parameter
- DataLoader digunakan untuk memuat data secara bertahap dalam bentuk mini-batch, mendukung proses shuffle data pada setiap epoch
- Optimizer seperti Adam menyediakan aturan pembaruan parameter yang sudah dioptimalkan, sehingga tidak perlu mengimplementasikan gradient descent dari nol

Metode dan Media Pembelajaran

Bentuk & Metode

Ceramah, praktikum dengan PyTorch, dan diskusi konsep

Media & Sumber Belajar

PyTorch, Google Colab

Estimasi Waktu

Kuliah (Praktikum): PB 180' + Praktik tatap muka 90' + PT 60' + BM 60' → Total 390 menit (~6,5 jam)

Evaluasi dan Tugas

Indikator Penilaian

- Mahasiswa mampu membuat model Neural Network dasar dan melakukan tuning

Bentuk & Kriteria Tugas

- Tugas implementasi model Neural Network dasar (struktur, akurasi, regularisasi)

Bobot penilaian pertemuan ini: 5%

Ringkasan Pertemuan

- Membentuk Model Non-Linear
- Arsitektur Jaringan Saraf
- Fungsi Aktivasi
- Keterbatasan Model Linier dan Fixed Basis Function
- Dari Neuron Biologis ke Neuron Artifisial
- Universal Approximation dan Mengapa 'Deep'?
- Membangun Neural Network dengan PyTorch
- Melatih Model dengan Autograd dan Optimizer

Pertemuan berikutnya akan melanjutkan pembahasan sesuai Rencana Pembelajaran Semester (RPS).

Referensi

Sumber Utama

- Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow (3rd ed.), O'Reilly, 2022
- Rahmad Syah, Pengantar Machine Learning: Teori dan Aplikasi dengan Python, Andi, 2021

Sumber Pendukung

- Andriy Burkov, The Hundred-Page Machine Learning Book, 2019
- Ian Goodfellow et al., Deep Learning, MIT Press, 2016
- Jake VanderPlas, Python Data Science Handbook, O'Reilly, 2016

Materi diperkaya dari CS189/289 UC Berkeley - Lecture 14: Neural Networks and PyTorch.